

DEFENSE DETECTION & EVASION METHODOLOGY

A Junior Penetration Tester's Field Notes on Load Balancers,
Web Application Firewalls, Network Firewalls & Antivirus Recon

Built from your WAFW00F / Nmap lab session (itpro.tv) + IT Pro TV lecture transcript —
reorganized into a structured study guide with practical commands, interpretation notes,
and detection strategy for each defense layer.

LEVEL: Junior / Foundational | SCOPE: Authorized lab targets only

How to Use These Notes

Each section below follows the same pattern: **what it is** → **why a pentester cares** → **how to detect it** (with real commands) → **how to read the output** → **detection/evasion strategy**. Work through them in order — this mirrors the actual order you hit these layers in a real engagement: recon the network path (load balancer), then the application edge (WAF), then the perimeter (firewall), then the endpoint (antivirus).

■ CAUTION — Scope reminder

Every command in this guide must only be run against systems you are explicitly authorized to test (a lab box, a CTF target, or a client in-scope asset with signed authorization). Running these tools against systems you don't own or have permission for is illegal in most jurisdictions.

The master recon flow

```
Target domain
↓
DNS lookup (nslookup / dig) – reveals multiple IPs → hints at load balancing
↓
Nmap port/service scan (80, 443) – confirms live web service(s)
↓
HTTP header analysis (curl -I) – server banner, CDN headers, security headers
↓
WAF fingerprinting (wafw00f, nmap http-waf-detect, cookies)
↓
Firewall / perimeter checks (firewalk, nmap evasion scripts)
↓
Endpoint defense awareness (antivirus indicators)
↓
Only THEN – proceed to authorized exploitation testing
```

SECTION 1

Load Balancers

A traffic-distribution layer that doubles as a defense mechanism

1.1 What it is (plain English)

A load balancer sits in front of a group of servers and spreads incoming traffic across them so no single server gets overwhelmed. From the outside you just see one address, e.g. `www.example.com` — you never know (or need to know) which of the 3, 5, or 50 backend servers actually answered your request.

CompTIA classifies load balancers as a defensive control because they help a system survive high traffic and Denial-of-Service (DoS) conditions — if one node is hit hard, traffic simply shifts to the others.

1.2 Why a pentester cares

- You might be hitting a *different backend server* on every request — inconsistent results (different response times, slightly different headers, session issues) can be a load-balancing artifact, not a bug in your attack.
- Load balancers are commonly bundled with WAF capability (e.g. AWS Elastic Load Balancer / ALB has native WAF integration) — so detecting one is often your first clue a WAF is nearby too.
- Knowing there's a pool of backend IPs means there may be a way to reach one directly, bypassing the balancer/edge protections entirely (see Strategy below).

1.3 Detection — practical commands

Step 1: DNS enumeration. Multiple A records returned for one hostname is your first signal.

COMMAND

```
nslookup itpro.tv
```

ALTERNATIVE (dig)

```
dig itpro.tv +noall +answer
```

■ ATTACKER MINDSET

Before you run it — predict: if this is a large SaaS product served globally, do you expect ONE IP or SEVERAL? Write your reasoning down in one line before scrolling to the output. This habit — predict, then verify — is what separates methodical testers from button-mashers.

If you see several unrelated-looking IPs (or IPs that resolve to a known cloud provider's ASN, e.g. AWS), that's evidence of load balancing / a CDN pool, not proof by itself.

Step 2: Confirm with header/banner grabbing.

COMMAND

```
curl -I https://itpro.tv/  
nmap -sV -p 80,443 itpro.tv
```

During Nmap's version/OS detection (banner grabbing), load-balancer software sometimes leaks itself directly — e.g. an F5 BIG-IP banner. Read every line of Nmap's service output; the label you need is often sitting in plain text.

Step 3: WAFW00F cross-check. Because ELB/ALB and similar load balancers often have built-in WAF signatures, wafw00f can indirectly reveal the load balancer:

COMMAND

```
wafw00f https://itpro.tv
```

1.4 Interpreting results

Evidence	What it tells you	What it does NOT tell you
Multiple A records in DNS	Traffic is likely distributed across >1 backend	Which balancing algorithm, or exact node count
Server: nginx` + AWS-owned IP range	Nginx may be the reverse proxy tier in front of app servers	Whether nginx itself is the LB, or just a downstream proxy
wafw00f reports "Elastic Load Balancer"	AWS ELB/ALB is in the request path, and may carry WAF rules	The exact rule set or whether WAF mode is even enabled

1.5 Detection Strategy (checklist)

- Always run DNS enumeration first — it's free, silent, and sets your expectations for everything after.
- Cross-reference IPs against known cloud ASNs (AWS, Azure, GCP, Cloudflare, Fastly, Vercel) using `whois` or `ipinfo.io` — this tells you the platform, which often tells you the likely LB/WAF stack.
- Note response-time and header variance across repeated requests — inconsistency across identical requests is a soft indicator of multiple backend nodes.
- Record everything as a hypothesis, not a fact, until you have two independent pieces of evidence pointing the same way (this is the 'evidence-based reconnaissance' principle from your lab session).

1.6 Evasion / Testing Strategy

Genuine "bypass" of a load balancer is rarely the goal — but there is one legitimate technique worth knowing:

■ **NOTE — Direct-to-origin testing**

If you can identify one of the actual backend IPs behind the load balancer/CDN (e.g. via historical DNS records, SSL certificate transparency logs, or a misconfigured subdomain that points directly at an origin server), you can sometimes send requests straight to that IP with the `Host` header manually set. This is useful for testing whether protections (WAF, rate limiting) that live only at the edge/CDN layer are actually enforced at the origin too — a very common real-world misconfiguration. It is not an "attack" on the load balancer itself, it's a way to test whether your edge security actually extends all the way back to the server.

SECTION 2

Web Application Firewalls (WAF)

Inspecting and filtering traffic to/from the web app itself

2.1 What it is (plain English)

A WAF sits between the client and the web application and inspects each HTTP/HTTPS request against a rule set / signature list. It's application-aware — unlike a generic network firewall, it understands things like SQL syntax, script tags, and malformed parameters.

```
Attacker
↓
CDN / Edge (e.g. CloudFront)
↓
WAF — inspects request:
normal request → ALLOW
suspicious pattern → BLOCK (commonly HTTP 403)
↓
Web server (e.g. nginx)
↓
Application
```

■ CAUTION — Don't over-conclude on a 403

A single HTTP 403 response does NOT by itself prove a WAF blocked you. It could be an auth failure, a misconfigured vhost, or the app itself returning 403. Treat it as one data point, not a verdict.

2.2 Why detect a WAF before testing further?

You want to understand the target's defensive architecture *before* you start sending attack payloads. Blind payload-spraying against an unknown WAF wastes time, generates noisy logs, and can trip alerting/lockout mechanisms before you've learned anything useful.

```
Discover target → Identify CDN → Identify possible WAF →
Understand filtering behavior → Continue authorized testing
```

2.3 Detection method #1 — WAFW00F

COMMAND (explore options first)

```
wafw00f --help
```

Flag	What it does
-v	Increase verbosity — more raw detail in the output (harder to read, more complete)

Flag	What it does
-a	Find ALL matching WAF signatures instead of stopping at the first match
-r	Skip following redirects — test the URL exactly as given
-p	Route the scan through a proxy (e.g. Burp) so you can inspect requests/responses
COMMAND — run against your lab target wafw00f https://itpro.tv	

Example result from the live lab run:

```

OUTPUT
[+] The site https://itpro.tv is behind Vercel WAF (Vercel) WAF.

```

2.4 Reading the result correctly

If wafw00f says...	Correct conclusion to write in your notes
A specific WAF is identified (e.g. Cloudflare, Vercel)	"WAFW00F identified a <vendor> WAF signature."
No WAF detected	"WAFW00F did not identify a known WAF signature." (NOT "there is no WAF" — detection tools aren't perfect, and some WAFs are tuned to stay invisible)

2.5 Detection method #2 — Nmap's WAF script

```

COMMAND
nmap -p 80,443 --script http-waf-detect <target-hostname>

```

Breaking the command down piece by piece:

Piece	Meaning
nmap	The scanner itself
-p 80,443	Only check the HTTP and HTTPS ports
--script http-waf-detect	Load Nmap's dedicated WAF-detection NSE script
<target-hostname>	Use the domain name, not a raw CDN IP — the WAF is usually name/host-based, so scanning the IP directly can miss it entirely

In the lab lecture demo, this scan against itpro.tv came back showing CloudFront in the response path with no single clean WAF label — a reminder that Nmap's script is one more data point, not a final answer, especially against layered CDN/WAF stacks.

2.6 Detection method #3 — Manual header & cookie inspection

```

COMMAND
curl -I https://www.itpro.tv/

```

EXAMPLE OUTPUT

```
HTTP/2 301
server: nginx
x-frame-options: sameorigin
content-security-policy: ...
via: ...cloudfront.net
x-cache: Miss from cloudfront
```

This is **direct evidence** — it's text you can literally read in the response. Compare it against **tool-based detection** (wafw00f's signature match), which is an inference, not a literal statement from the server.

■ ATTACKER MINDSET

In the real lab run, wafw00f reported "Vercel WAF" but the curl headers only showed nginx + CloudFront — no mention of Vercel anywhere. Question for you: does that mean wafw00f was wrong? (Answer: not necessarily — signature-based tools detect fingerprints that don't always surface in visible headers. It means you write down the discrepancy and treat the WAF identity as unconfirmed until a second source agrees.)

Also check cookies — some WAFs inject their own tracking/session cookie (e.g. names containing `incap_ses`, `visid_incap` for Imperva, or `AWSALB/awse lb` for AWS). Use your browser dev tools or `curl -v` and read the `Set-Cookie` lines.

2.7 The core evidence-based reconnaissance principle

```
Tool says X
↓
Do NOT write: "X is definitely true"
↓
Look for a second, independent piece of supporting evidence
↓
Confirm / weaken / reject the hypothesis
```

Professional note-writing example from a real finding: *"WAFW00F identified a Vercel WAF signature, but the HTTP response headers do not explicitly expose Vercel. Further validation would be required before treating the exact WAF/infrastructure chain as confirmed."* This is the tone and rigor expected in a professional pentest report — confident about what you directly observed, cautious about what you only inferred.

2.8 WAF Detection Strategy (checklist)

- Run wafw00f with default settings first — fast triage.
- If it finds nothing or something ambiguous, re-run with `-a` to catch layered/multiple WAFs.
- Always cross-check with a manual `curl -I` and a look at `Set-Cookie` headers.
- Run the Nmap `http-waf-detect` script against the hostname (not the bare IP).
- Log every finding with its evidence type: **Direct** (headers/cookies you can literally read) vs. **Tool-inferred** (signature match) — never blend the two into a single unqualified claim.

2.9 WAF evasion — theory you should know (conceptual only)

WAFs work off pattern/signature matching, so the general bypass principle taught at this level is: **if a payload's exact pattern isn't on the WAF's blacklist, it may pass through even though it's functionally the same attack.** Common

categories you'll study later in your training (in a proper authorized lab with a specific vulnerable target) include:

- **Encoding tricks** — URL-encoding, double-encoding, or Base64-wrapping parts of a payload so the literal blocked string never appears on the wire.
- **Case and whitespace manipulation** — many older/weaker WAF rules are case-sensitive or expect exact spacing, so varying case or inserting harmless whitespace/comments can slip past a narrow rule.
- **Alternate syntax** — using a functionally-equivalent but differently-worded version of a request that achieves the same result the WAF didn't anticipate.

■ **CAUTION — Where this guide draws the line**

The specific payload strings and step-by-step bypass sequences for a given vulnerability class (SQLi, XSS, etc.) are technique-specific and target-specific — they belong in your hands-on lab modules for that vulnerability (with your instructor / OSCP-style material), not in a general reference doc. The principle to remember for the exam and for real work: a WAF's effectiveness is only as good as its specific rule list, so “find what's outside the list” is the mindset — not a fixed set of tricks.

SECTION 3

Network Firewalls

Perimeter ACL enforcement — detection via firewalking & Nmap evasion

3.1 What it is (plain English)

A traditional (network-layer) firewall enforces Access Control Lists (ACLs): a request either matches an “allow” rule and passes, or it doesn't and gets dropped/rejected. Unlike a WAF, it typically isn't inspecting application-layer content — it's working off IPs, ports, and protocols.

3.2 Detection method #1 — Firewalking

Firewalking maps which ports are reachable *past* a firewall by sending packets with a TTL (time-to-live) set to expire exactly one hop beyond the firewall. If the packet gets through the firewall and then expires on the far side, you get a “Time Exceeded” ICMP message — proof the packet made it past the filtering device before dying naturally. If it never gets that far, you learn the firewall dropped it outright.

SETUP

```
sudo apt install firewalk
firewalk --help
```

COMMAND (concept)

```
firewalk -pTCP -d 80 <gateway-ip> <target-ip-behind-it>
```

Nmap ships an equivalent NSE script:

LOCATE THE SCRIPT

```
find / -name '*.nse' | grep -i firewalk
```

COMMAND

```
nmap --script firewalk <target>
```

3.3 Detection method #2 — Nmap evasion/spoofing flags

Nmap has a family of flags built specifically to probe how a firewall reacts to unusual packet shapes. Understanding them is as much about detection as evasion — if a scan behaves differently with these flags versus without, you've learned something about the filtering device in the path.

Flag	What it does	Why it matters
-f	Fragments probe packets into smaller pieces	Some older/weaker firewalls can't reassemble and inspect fragments properly, so a fragmented scan can slip through while a normal one is blocked

Flag	What it does	Why it matters
<code>-D decoy1, decoy2, ME</code>	Sends the scan mixed in with spoofed decoy source IPs	Makes it harder for the firewall/IDS to know which source IP is the real scanner — useful for testing detection/alerting, not guaranteed to bypass filtering rules
<code>-g <port> / --source-port</code>	Sends probes from a specific source port (e.g. 53 or 80)	Some ACLs trust traffic from “well-known” source ports and let it through unfiltered
<code>--data-length <n></code>	Pads packets with extra junk data	Can help probes look less like default Nmap traffic to a signature-based IDS/firewall

■ **CAUTION — Evasion flags are diagnostic, not magic**

None of these guarantee a bypass against a modern, well-configured firewall. Their real value at your level is comparative: run a scan without them, then with them, and compare results. A difference tells you something concrete about how the filtering device is configured.

3.4 Firewall Detection Strategy (checklist)

- Start with a normal Nmap scan to get a baseline (open/closed/filtered ports).
- Re-run key probes with `-f` and compare — new ports showing up as open is a strong signal.
- Use firewalking (tool or Nmap script) specifically when you need to know which ports are reachable on a host *behind* a known gateway/firewall, not just whether the firewall itself is there.
- Document baseline vs. evasion-flag results side by side in your notes — the delta IS the finding.

SECTION 4

Antivirus / Endpoint Defenses

Detecting presence — without building evasive payloads

4.1 What it is (plain English)

Antivirus (AV) / endpoint protection inspects files, processes, and sometimes network traffic on a host for known-malicious signatures and suspicious behavior. On any modern Windows host you should simply assume Windows Defender (or a third-party product) is present and running unless you find specific evidence otherwise.

4.2 Legitimate ways to detect AV presence

- **Default assumption:** Windows Defender is on by default on modern Windows — treat “no AV” as the exception you have to prove, not the default you assume.
- **EICAR test file:** the industry-standard, harmless test string published for exactly this purpose. Dropping the official EICAR test file onto a host and observing whether it's immediately quarantined is the safe, standard way to confirm AV is active — it is not malware and triggers on the signature alone.
- **Network traffic patterns:** AV products phone home to update — traffic to known AV vendor domains/IP ranges (e.g. Kaspersky, CrowdStrike, SentinelOne update servers) is a strong indicator of which product is installed.
- **Running services / process list** (post-compromise, with authorization): service names, scheduled tasks, and installed-program lists will usually name the AV product directly.
- **Social engineering (within engagement scope):** casual questions to staff about “what antivirus pop-up did you get” or “did your AV just update” can reveal product and version without ever touching a system.
- **Documentation review:** IT asset inventories, security policy documents, or onboarding memos obtained legitimately during the engagement often state the AV product outright.

■ CAUTION — What this guide will not teach

Your source lecture also demonstrated building an obfuscated, fileless PowerShell payload (base64-encoded, in-memory execution) specifically to evade antivirus signatures. That's payload/malware construction, not detection methodology, and it's outside what these notes cover — that kind of technique needs to live inside a formal, supervised red-team curriculum with explicit rules of engagement, not a self-study PDF. At your current level, focus on detecting AV and reasoning about why a given off-the-shelf tool got flagged; payload development comes much later and under direct instructor supervision.

4.3 What to actually take away at junior level

- AV detection is about gathering evidence quietly and correctly attributing it — same evidence-based mindset as WAF detection.
- Understand at a conceptual level *why* signature-based AV can be evaded (unknown/novel patterns, in-memory execution, encoding) so you can reason about findings in a report — without needing to build such payloads yourself yet.
- In a professional engagement, endpoint testing is usually scoped and supervised separately from web recon — know the boundary between the two engagement types.

SECTION 5

Consolidated Cheat Sheet

Quick reference for lab day

5.1 One command per defense layer

Layer	Fastest check	What confirms it
Load balancer	<code>nslookup <target></code>	Multiple A records / known cloud ASN
WAF	<code>wafw00f https://<target></code>	Named vendor signature in output
WAF (cross-check)	<code>curl -I https://<target></code>	Server/CDN/security headers, Set-Cookie names
WAF (deep check)	<code>nmap -p 80,443 --script http-waf-detect <host></code>	Script output flags a filtering layer
Firewall	<code>nmap --script firewalk <target></code>	Time-Exceeded response one hop past the gateway
Firewall (evasion test)	<code>nmap -f <target></code>	Different result vs. baseline scan
Antivirus	Drop the official EICAR test file	Immediate quarantine/deletion

5.2 Junior-level lab exercise checklist

- [] Run `nslookup` against your authorized lab target and record every IP returned.
- [] Run `wafw00f` with default settings, then again with `-a`.
- [] Run `curl -I` and list every header that could indicate CDN/WAF/LB — quote them exactly.
- [] Run `nmap --script http-waf-detect` against the hostname and compare to the `wafw00f` result.
- [] Write one paragraph in “professional note” tone stating what's confirmed vs. what's only inferred.
- [] Run a baseline Nmap scan, then re-run with `-f` — note any difference in reported port states.
- [] Explain, in your own words, why a 403 response alone is not proof of a WAF.

5.3 The single mindset rule to carry forward

■ ATTACKER MINDSET

Every tool gives you a hypothesis, not a verdict. Your job as a pentester isn't to run tools — it's to run tools, then interrogate the results against a second, independent source before you write anything down as fact. That habit is worth more than memorizing any single command in this guide.

Source material: your live WAFW00F/Nmap lab session against *itpro.tv*, and the IT Pro TV lecture transcript on defense detection and evasion. Compiled and expanded into a structured study guide.